Multi-Variate Models

Ray Chandonnet and Barrett Viator

2025-08-13

```
# Import Libraries

library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(dplyr)
library(readxl)
library(writexl)
library(skimr)
library(knitr)
library(kableExtra)
```

```
##
## Attaching package: 'kableExtra'
##
## The following object is masked from 'package:dplyr':
##
##     group_rows
```

```
library(tidyr)
library(ggpmisc)
```

```
## Loading required package: ggpp
## Registered S3 methods overwritten by 'ggpp':
##   method                  from
##   heightDetails.titleGrob ggplot2
##   widthDetails.titleGrob  ggplot2
##
## Attaching package: 'ggpp'
##
## The following object is masked from 'package:ggplot2':
##
##     annotate
```

```
library(forcats)
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(naniar)
```

```
##
## Attaching package: 'naniar'
##
## The following object is masked from 'package:skimr':
##
##     n_complete
```

```
library(reshape2)
```

```
##
## Attaching package: 'reshape2'
##
## The following object is masked from 'package:tidyr':
##
##     smiths
```

```
library(scales)
```

```
##
## Attaching package: 'scales'
##
## The following object is masked from 'package:purrr':
##
##     discard
##
## The following object is masked from 'package:readr':
##
##     col_factor
```

```
library(stringr)
library(recipes)
```

```
##
## Attaching package: 'recipes'
##
## The following object is masked from 'package:stringr':
##
##     fixed
##
## The following object is masked from 'package:stats':
##
##     step
```

```
library(broom)
library(here)
```

```
## here() starts at /Users/raychandonnet/Chandonnet Family Dropbox/Chandonnet Family Folder/Ray Personal
```

```
library(rlang)
```

```
##
```

```
## Attaching package: 'rlang'
##
## The following objects are masked from 'package:purrr':
##
##     %@%, flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
##     flatten_raw, invoke, splice
```

```
library(rsample)
library(xts)
```

```
## Loading required package: zoo
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
##
##
## ############################ Warning from 'xts' package ##########################
## #                                                                                #
## # The dplyr lag() function breaks how base R's lag() function is supposed to  #
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or       #
## # source() into this session won't work correctly.                               #
## #                                                                                #
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop           #
## # dplyr from breaking base R's lag() function.                                   #
## #                                                                                #
## # Code in packages is not affected. It's protected by R's namespace mechanism #
## # Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning.  #
## #                                                                                #
## ##################################################################################
##
## Attaching package: 'xts'
##
## The following objects are masked from 'package:dplyr':
##
##     first, last
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```r
library(caTools)
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:rsample':
##
##     calibration
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(RANN)
library(xgboost)
```

```
##
## Attaching package: 'xgboost'
##
## The following object is masked from 'package:dplyr':
##
##     slice
```

```r
library(Matrix)
```

```
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```r
library(e1071)
```

```
##
## Attaching package: 'e1071'
##
## The following object is masked from 'package:rsample':
##
##     permutations
```

```r
# Read in the saved training and test data sets
train_data <- readRDS(here::here("Data", "train_data.rds"))
test_data <- readRDS(here::here("Data", "test_data.rds"))
PCAData_test <- readRDS(here::here("Data", "PCAData_test.rds"))
PCAData_train <- readRDS(here::here("Data", "PCAData_train.rds"))
importance_matrix <- readRDS(here::here("Data", "xgboost_importance_matrix.rds"))

# This first model is an XGBoost model.  It will predict the HDI category, not the
# index, and it uses static current InternetUsersPct as our "key" feature

model_columns <- c(
  "CountryCode", "HDI_Category", "FoodIndex", "ElectricAccess", "PopDensity",
  "PopInSlums", "WaterStress", "InternetUsersPct", "GDPPerCapGrowth", "GDPGrowth",
```

```
  "GDP", "PoliticalStability", "RqdEduYears", "GovtEduSpendPctGDP",
  "UHCServiceCoverage", "RuralPopulGrowth", "VoiceAccountability"
  )

# Filter the training and test data to include only the specified columns

train_filtered <- train_data %>% dplyr::select(all_of(model_columns))
test_filtered <- test_data %>% dplyr::select(all_of(model_columns))

# Use na.omit() to remove rows with any NA values

train_filtered <- na.omit(train_filtered)
test_filtered <- na.omit(test_filtered)

# Keep a country code vector from training data (for constructing folds)
train_ids <- train_filtered$CountryCode   # same row order as train_matrix / dtrain

# Remove the country code from the predictor data frames
train_predictors <- train_filtered %>% dplyr::select(-CountryCode)
test_predictors <- test_filtered %>% dplyr::select(-CountryCode)

# For classification, the target variable must be a numeric integer
HDI_levels <- c(
  "Low human development",
  "Medium human development",
  "High human development",
  "Very high human development"
)

train_target <- as.integer(factor(train_filtered$HDI_Category, levels = HDI_levels)) - 1
test_target <- as.integer(factor(test_filtered$HDI_Category, levels = HDI_levels)) - 1
num_class <- length(unique(train_target))

# Remove the target variable from the predictor data frames
train_predictors <- train_predictors %>% dplyr::select(-HDI_Category)
test_predictors <- test_predictors %>% dplyr::select(-HDI_Category)

# Combine the predictor data frames to create a sparse matrix that has the same
# columns and column order for both training and test data.

combined_predictors <- bind_rows(train_predictors, test_predictors)

# Convert all factor/character columns to a sparse matrix for XGBoost

combined_matrix <- sparse.model.matrix(~ . -1, data = combined_predictors)


# Split the combined matrix back into training and test matrices

train_matrix <- combined_matrix[1:nrow(train_predictors),]
test_matrix <- combined_matrix[(nrow(train_predictors)+1):nrow(combined_matrix),]

# Train XGBoost Model with hyperparameter Tuning and cross-validation
```

```r
dtrain <- xgb.DMatrix(data = train_matrix, label = train_target)
dtest <- xgb.DMatrix(data = test_matrix, label = test_target)

# Define a grid of hyperparameters to search over

hyper_grid <- expand.grid(
  eta = c(0.01, 0.1, 0.3),
  max_depth = c(6, 12, 18),
  subsample = c(0.6, 0.8, 1.0)
)

# Create an empty list to store the results

results_list <- list()

# We are going to Iterate over each combination of hyperparameters.  The model uses
# k fold cross-validation - Because this is temporal data and so observations for a
# given country are autocorrelated, we need to make sure that whole countries are
# contained within folds, much as we did with the test/training split.
#
# We also use the same folds for each iteration so that when selecting the best set of
# hyperparameters we are comparing apples to apples

# Create the folds by country
k <- 5
set.seed(123)
countries <- unique(train_ids)
fold_id <- sample(rep(1:k, length.out = length(countries)))
folds <- lapply(1:k, function(j) which(train_ids %in% countries[fold_id == j]))

# Set maximum threads to avoid crashing
Sys.setenv(OMP_NUM_THREADS="1", MKL_NUM_THREADS="1")

# Run and evaluate model for each set of hyperparameters
for (i in 1:nrow(hyper_grid)) {
  current_eta <- hyper_grid$eta[i]
  current_max_depth <- hyper_grid$max_depth[i]
  current_subsample <- hyper_grid$subsample[i]

  params <- list(
    booster = "gbtree",
    objective = "multi:softprob",
    num_class = num_class,
    eta = current_eta,
    max_depth = current_max_depth,
    subsample = current_subsample,
    colsample_bytree = 0.8,
    nthread = 1,
    seed = 123
  )

# We use xgb.cv for hyperparameter tuning.
```

```r
  cv_model <- xgb.cv(
    params = params,
    data = dtrain,
    nrounds = 3000,
    folds = folds,
    metrics = "mlogloss",
    early_stopping_rounds = 10,
    verbose = 0
  )

# Get the best cross-validation accuracy

  best_iteration <- cv_model$best_iteration
  cv_loglossmean <- cv_model$evaluation_log[best_iteration,]$test_mlogloss_mean
  roundsrun <- nrow(cv_model$evaluation_log)

# Store the results

  results_list[[i]] <- list(
    eta = current_eta,
    max_depth = current_max_depth,
    roundsrun = roundsrun,
    subsample = current_subsample,
    loglossmean = cv_loglossmean,
    best_iteration = best_iteration
  )

  cat(paste0("Model ", i, " of ",nrow(hyper_grid),
             ", Settings: eta=", current_eta,
             ", max_depth=", current_max_depth,
             ", subsample=", current_subsample,
             ", Rounds Run=", roundsrun,
             " | CV Logloss=", round(cv_loglossmean, 4),
             " (Best Iteration: ", best_iteration, ")\n"))

}  #Closes For Loop
```

```
## Model 1 of 27, Settings: eta=0.01, max_depth=6, subsample=0.6, Rounds Run=403 | CV Logloss=0.6039 (Be
## Model 2 of 27, Settings: eta=0.1, max_depth=6, subsample=0.6, Rounds Run=44 | CV Logloss=0.6001 (Best
## Model 3 of 27, Settings: eta=0.3, max_depth=6, subsample=0.6, Rounds Run=23 | CV Logloss=0.6254 (Best
## Model 4 of 27, Settings: eta=0.01, max_depth=12, subsample=0.6, Rounds Run=421 | CV Logloss=0.6116 (B
## Model 5 of 27, Settings: eta=0.1, max_depth=12, subsample=0.6, Rounds Run=45 | CV Logloss=0.6118 (Bes
## Model 6 of 27, Settings: eta=0.3, max_depth=12, subsample=0.6, Rounds Run=21 | CV Logloss=0.6489 (Bes
## Model 7 of 27, Settings: eta=0.01, max_depth=18, subsample=0.6, Rounds Run=423 | CV Logloss=0.6116 (B
## Model 8 of 27, Settings: eta=0.1, max_depth=18, subsample=0.6, Rounds Run=50 | CV Logloss=0.6214 (Bes
## Model 9 of 27, Settings: eta=0.3, max_depth=18, subsample=0.6, Rounds Run=23 | CV Logloss=0.6417 (Bes
## Model 10 of 27, Settings: eta=0.01, max_depth=6, subsample=0.8, Rounds Run=403 | CV Logloss=0.612 (Be
## Model 11 of 27, Settings: eta=0.1, max_depth=6, subsample=0.8, Rounds Run=46 | CV Logloss=0.6134 (Bes
## Model 12 of 27, Settings: eta=0.3, max_depth=6, subsample=0.8, Rounds Run=23 | CV Logloss=0.6194 (Bes
## Model 13 of 27, Settings: eta=0.01, max_depth=12, subsample=0.8, Rounds Run=391 | CV Logloss=0.6206
## Model 14 of 27, Settings: eta=0.1, max_depth=12, subsample=0.8, Rounds Run=49 | CV Logloss=0.6363 (Be
## Model 15 of 27, Settings: eta=0.3, max_depth=12, subsample=0.8, Rounds Run=22 | CV Logloss=0.6601 (Be
## Model 16 of 27, Settings: eta=0.01, max_depth=18, subsample=0.8, Rounds Run=386 | CV Logloss=0.6195
## Model 17 of 27, Settings: eta=0.1, max_depth=18, subsample=0.8, Rounds Run=48 | CV Logloss=0.6257 (Be
```

```
## Model 18 of 27, Settings: eta=0.3, max_depth=18, subsample=0.8, Rounds Run=24 | CV Logloss=0.646 (Bes
## Model 19 of 27, Settings: eta=0.01, max_depth=6, subsample=1, Rounds Run=382 | CV Logloss=0.6273 (Bes
## Model 20 of 27, Settings: eta=0.1, max_depth=6, subsample=1, Rounds Run=51 | CV Logloss=0.6208 (Best
## Model 21 of 27, Settings: eta=0.3, max_depth=6, subsample=1, Rounds Run=22 | CV Logloss=0.6438 (Best
## Model 22 of 27, Settings: eta=0.01, max_depth=12, subsample=1, Rounds Run=365 | CV Logloss=0.6517 (Be
## Model 23 of 27, Settings: eta=0.1, max_depth=12, subsample=1, Rounds Run=44 | CV Logloss=0.6503 (Best
## Model 24 of 27, Settings: eta=0.3, max_depth=12, subsample=1, Rounds Run=23 | CV Logloss=0.6619 (Best
## Model 25 of 27, Settings: eta=0.01, max_depth=18, subsample=1, Rounds Run=377 | CV Logloss=0.6396 (Be
## Model 26 of 27, Settings: eta=0.1, max_depth=18, subsample=1, Rounds Run=46 | CV Logloss=0.6599 (Best
## Model 27 of 27, Settings: eta=0.3, max_depth=18, subsample=1, Rounds Run=22 | CV Logloss=0.6699 (Best
```

```r
# Summarize results
results_df <- do.call(rbind, lapply(results_list, as.data.frame))
best_model_settings <- results_df[which.min(results_df$loglossmean),]

cat("\n--- Best Model Settings (Based on CV Logloss) ---\n")
```

```
##
## --- Best Model Settings (Based on CV Logloss) ---
```

```r
print(best_model_settings)
```

```
##   eta max_depth roundsrun subsample loglossmean best_iteration
## 2 0.1         6        44       0.6   0.6001259             34
```

```r
cat("--------------------------\n")
```

```
## --------------------------
```

```r
set.seed(123)

# Final Model with best settings
params_final <- list(
  booster = "gbtree",
  objective = "multi:softprob",
  num_class = num_class,
  eta = best_model_settings$eta,
  max_depth = best_model_settings$max_depth,
  subsample = best_model_settings$subsample,
  colsample_bytree = 0.8,
  nthread = 1
)

final_xgb_model <- xgboost(
  params = params_final,
  data = train_matrix,
  label = train_target,
  nrounds = best_model_settings$best_iteration,
  verbose = 1
)
```

```
## [1]  train-mlogloss:1.242981
## [2]  train-mlogloss:1.125314
## [3]  train-mlogloss:1.022646
## [4]  train-mlogloss:0.933665
## [5]  train-mlogloss:0.855877
## [6]  train-mlogloss:0.788349
```

```
## [7]  train-mlogloss:0.729536
## [8]  train-mlogloss:0.677812
## [9]  train-mlogloss:0.629544
## [10] train-mlogloss:0.584968
## [11] train-mlogloss:0.548079
## [12] train-mlogloss:0.513817
## [13] train-mlogloss:0.481124
## [14] train-mlogloss:0.453500
## [15] train-mlogloss:0.427012
## [16] train-mlogloss:0.402071
## [17] train-mlogloss:0.380486
## [18] train-mlogloss:0.361065
## [19] train-mlogloss:0.341037
## [20] train-mlogloss:0.321606
## [21] train-mlogloss:0.303469
## [22] train-mlogloss:0.287295
## [23] train-mlogloss:0.272875
## [24] train-mlogloss:0.259800
## [25] train-mlogloss:0.246644
## [26] train-mlogloss:0.236163
## [27] train-mlogloss:0.226125
## [28] train-mlogloss:0.217116
## [29] train-mlogloss:0.207564
## [30] train-mlogloss:0.197887
## [31] train-mlogloss:0.187852
## [32] train-mlogloss:0.179876
## [33] train-mlogloss:0.172207
## [34] train-mlogloss:0.165118
```

```
# Make predictions on the test data with the final model

predictions_final <- predict(final_xgb_model, newdata = test_matrix)
predictions_class_final <- matrix(predictions_final, ncol = num_class, byrow = TRUE) %>%
  apply(1, which.max) - 1

# Calculate the Accuracy

final_accuracy <- sum(predictions_class_final == test_target) / length(test_target)
cat("\nFinal XGBoost Accuracy on Test Data:", final_accuracy, "\n")
```

```
##
## Final XGBoost Accuracy on Test Data: 0.7669991
```

```
# Visualize model performance

confusion_matrix <- table(Actual = test_target, Predicted = predictions_class_final)
print("Confusion Matrix:")
```

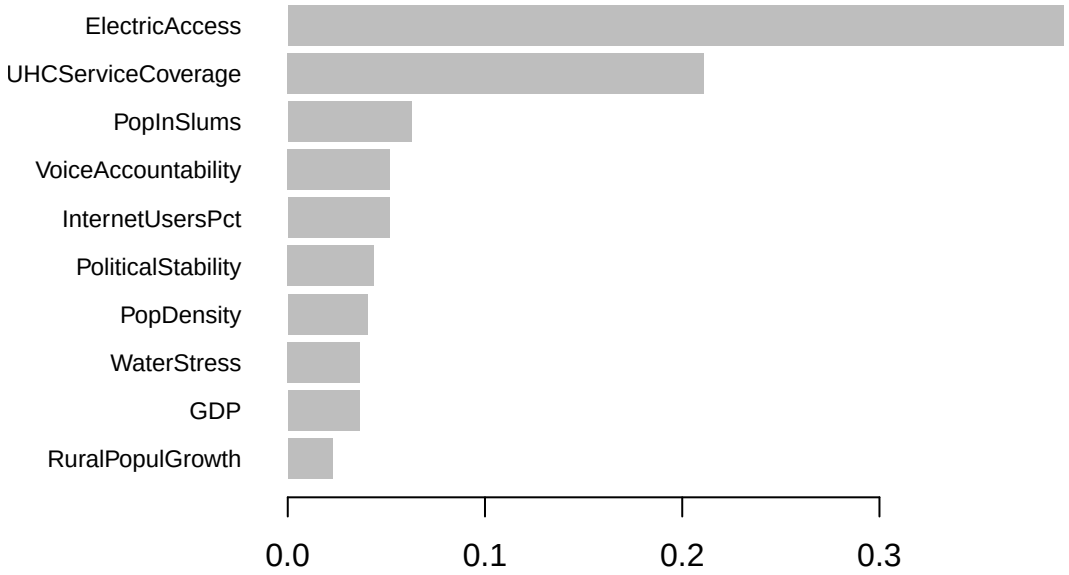
```
## [1] "Confusion Matrix:"
```

```
print(confusion_matrix)
```

```
##       Predicted
## Actual   0   1   2   3
##      0 203  67   0   0
##      1  33 119  57   0
```

```
##       2   0  16 198  11
##       3   0   0  73 326
```

```
importance_matrix <- xgb.importance(feature_names = colnames(train_matrix), model = final_xgb_model)
xgb.plot.importance(importance_matrix, top_n = 10)
```




```
cat("\nSummary of Feature Importance Matrix:\n")
```

```
##
## Summary of Feature Importance Matrix:
```

```
print(importance_matrix)
```

```
##                 Feature        Gain       Cover  Frequency  Importance
##                  <char>       <num>       <num>      <num>       <num>
##  1:      ElectricAccess 0.393235910 0.212992694 0.10003301 0.393235910
##  2:  UHCServiceCoverage 0.210990904 0.157155984 0.08682734 0.210990904
##  3:          PopInSlums 0.062624060 0.084384203 0.09243975 0.062624060
##  4: VoiceAccountability 0.051778281 0.074601572 0.07725322 0.051778281
##  5:    InternetUsersPct 0.051573450 0.092143904 0.08616705 0.051573450
##  6:  PoliticalStability 0.043785436 0.055991516 0.06999010 0.043785436
##  7:          PopDensity 0.040371847 0.063271152 0.09574117 0.040371847
##  8:         WaterStress 0.036598212 0.049811657 0.07527237 0.036598212
##  9:                 GDP 0.036388223 0.051331235 0.07758336 0.036388223
## 10:    RuralPopulGrowth 0.022692594 0.041143043 0.05018158 0.022692594
## 11:  GovtEduSpendPctGDP 0.019269972 0.037638369 0.05909541 0.019269972
## 12:           FoodIndex 0.014076541 0.040189087 0.05975569 0.014076541
## 13:         RqdEduYears 0.010117561 0.016767180 0.02476065 0.010117561
## 14:     GDPPerCapGrowth 0.003696952 0.016890549 0.02740178 0.003696952
## 15:           GDPGrowth 0.002800056 0.005687854 0.01749752 0.002800056
```

```
saveRDS(importance_matrix, file = "xgboost_importance_matrix.rds")

##############################################################################

# XGBoost InternetUserPct Regression
```

```r
# Create a List of columns to evaluate

model_columns2 <- c(
  "CountryCode", "HDI_Index", "FoodIndex", "ElectricAccess", "PopDensity",
  "PopInSlums", "WaterStress", "InternetUsersPct", "GDPPerCapGrowth", "GDPGrowth",
  "GDP", "PoliticalStability", "RqdEduYears", "GovtEduSpendPctGDP",
  "UHCServiceCoverage", "RuralPopulGrowth", "VoiceAccountability"
)

# Filter the training and test data to include only the specified columns

train_filtered2 <- train_data %>% dplyr::select(all_of(model_columns2))
test_filtered2 <- test_data %>% dplyr::select(all_of(model_columns2))

# Use na.omit() to remove rows with any NA values

train_filtered2 <- na.omit(train_filtered2)
test_filtered2 <- na.omit(test_filtered2)

# Keep a country code vector from training data (for constructing folds)
train_ids2 <- train_filtered2$CountryCode   # same row order as train_matrix / dtrain

# Remove the country code from the predictor data frames
train_predictors2 <- train_filtered2 %>% dplyr::select(-CountryCode)
test_predictors2 <- test_filtered2 %>% dplyr::select(-CountryCode)

train_target2 <- train_filtered2$HDI_Index
test_target2 <- test_filtered2$HDI_Index

# Remove the target variable from the predictor data frames
train_predictors2 <- train_predictors2 %>% dplyr::select(-HDI_Index)
test_predictors2 <- test_predictors2 %>% dplyr::select(-HDI_Index)

# Combine the predictor data frames to create a sparse matrix that has the same
# columns and column order for both training and test data.

combined_predictors2 <- bind_rows(train_predictors2, test_predictors2)

# Convert all factor/character columns to a sparse matrix for XGBoost

combined_matrix2 <- sparse.model.matrix(~ . -1, data = combined_predictors2)


# Split the combined matrix back into training and test matrices

train_matrix2 <- combined_matrix2[1:nrow(train_predictors2),]
test_matrix2 <- combined_matrix2[(nrow(train_predictors2)+1):nrow(combined_matrix2),]

# Train XGBoost Model with hyperparameter Tuning and cross-validation

dtrain2 <- xgb.DMatrix(data = train_matrix2, label = train_target2)
dtest2 <- xgb.DMatrix(data = test_matrix2, label = test_target2)
```

```r
# Define a grid of hyperparameters to search over

hyper_grid2 <- expand.grid(
  eta = c(0.01, 0.1, 0.3),
  max_depth = c(6, 12, 18),
  subsample = c(0.6, 0.8, 1.0)
)

# Create an empty list to store the results

results_list2 <- list()

# We are going to Iterate over each combination of hyperparameters.  The model uses
# k fold cross-validation - Because this is temporal data and so observations for a
# given country are autocorrelated, we need to make sure that whole countries are
# contained within folds, much as we did with the test/training split.
#
# We also use the same folds for each iteration so that when selecting the best set of
# hyperparameters we are comparing apples to apples

# Create the folds by country
k <- 5
set.seed(123)
countries2 <- unique(train_ids2)
fold_id2 <- sample(rep(1:k, length.out = length(countries2)))
folds2 <- lapply(1:k, function(j) which(train_ids2 %in% countries2[fold_id2 == j]))

# Run and evaluate model for each set of hyperparameters
for (i in 1:nrow(hyper_grid2)) {
  current_eta <- hyper_grid2$eta[i]
  current_max_depth <- hyper_grid2$max_depth[i]
  current_subsample <- hyper_grid2$subsample[i]
  set.seed(123)
  params <- list(
    booster = "gbtree",
    objective = "reg:squarederror",
    eta = current_eta,
    max_depth = current_max_depth,
    subsample = current_subsample,
    colsample_bytree = 0.8,
    nthread = 1
  )

  # We use xgb.cv for hyperparameter tuning.

  cv_model <- xgb.cv(
    params = params,
    data = dtrain2,
    nrounds = 3000,
    folds = folds2,
    metrics = "rmse",
    early_stopping_rounds = 10,
    verbose = 0
```

```
  )

  # Get the best cross-validation accuracy

  best_iteration <- cv_model$best_iteration
  cv_rmse_mean <- cv_model$evaluation_log[best_iteration,]$test_rmse_mean
  roundsrun <- nrow(cv_model$evaluation_log)

  # Store the results

  results_list2[[i]] <- list(
    eta = current_eta,
    max_depth = current_max_depth,
    roundsrun = roundsrun,
    subsample = current_subsample,
    rmse = cv_rmse_mean,
    best_iteration = best_iteration
  )

  cat(paste0("Model ", i, " of ",nrow(hyper_grid2),
             ", Settings: eta=", current_eta,
             ", max_depth=", current_max_depth,
             ", subsample=", current_subsample,
             ", Rounds Run=", roundsrun,
             " | CV RMSE =", round(cv_rmse_mean, 4),
             " (Best Iteration: ", best_iteration, ")\n"))

}  #Closes For Loop
```

```
## Model 1 of 27, Settings: eta=0.01, max_depth=6, subsample=0.6, Rounds Run=650 | CV RMSE =0.0421 (Bes
## Model 2 of 27, Settings: eta=0.1, max_depth=6, subsample=0.6, Rounds Run=78 | CV RMSE =0.0427 (Best
## Model 3 of 27, Settings: eta=0.3, max_depth=6, subsample=0.6, Rounds Run=30 | CV RMSE =0.045 (Best I
## Model 4 of 27, Settings: eta=0.01, max_depth=12, subsample=0.6, Rounds Run=847 | CV RMSE =0.0426 (Be
## Model 5 of 27, Settings: eta=0.1, max_depth=12, subsample=0.6, Rounds Run=86 | CV RMSE =0.0435 (Best
## Model 6 of 27, Settings: eta=0.3, max_depth=12, subsample=0.6, Rounds Run=41 | CV RMSE =0.047 (Best
## Model 7 of 27, Settings: eta=0.01, max_depth=18, subsample=0.6, Rounds Run=916 | CV RMSE =0.0426 (Be
## Model 8 of 27, Settings: eta=0.1, max_depth=18, subsample=0.6, Rounds Run=125 | CV RMSE =0.0433 (Bes
## Model 9 of 27, Settings: eta=0.3, max_depth=18, subsample=0.6, Rounds Run=49 | CV RMSE =0.0467 (Best
## Model 10 of 27, Settings: eta=0.01, max_depth=6, subsample=0.8, Rounds Run=669 | CV RMSE =0.0423 (Be
## Model 11 of 27, Settings: eta=0.1, max_depth=6, subsample=0.8, Rounds Run=100 | CV RMSE =0.0431 (Bes
## Model 12 of 27, Settings: eta=0.3, max_depth=6, subsample=0.8, Rounds Run=29 | CV RMSE =0.047 (Best
## Model 13 of 27, Settings: eta=0.01, max_depth=12, subsample=0.8, Rounds Run=965 | CV RMSE =0.0428 (B
## Model 14 of 27, Settings: eta=0.1, max_depth=12, subsample=0.8, Rounds Run=108 | CV RMSE =0.0439 (Be
## Model 15 of 27, Settings: eta=0.3, max_depth=12, subsample=0.8, Rounds Run=35 | CV RMSE =0.0478 (Bes
## Model 16 of 27, Settings: eta=0.01, max_depth=18, subsample=0.8, Rounds Run=1169 | CV RMSE =0.0428 (
## Model 17 of 27, Settings: eta=0.1, max_depth=18, subsample=0.8, Rounds Run=165 | CV RMSE =0.044 (Bes
## Model 18 of 27, Settings: eta=0.3, max_depth=18, subsample=0.8, Rounds Run=44 | CV RMSE =0.0479 (Bes
## Model 19 of 27, Settings: eta=0.01, max_depth=6, subsample=1, Rounds Run=601 | CV RMSE =0.0429 (Best
## Model 20 of 27, Settings: eta=0.1, max_depth=6, subsample=1, Rounds Run=77 | CV RMSE =0.0439 (Best I
## Model 21 of 27, Settings: eta=0.3, max_depth=6, subsample=1, Rounds Run=31 | CV RMSE =0.0445 (Best I
## Model 22 of 27, Settings: eta=0.01, max_depth=12, subsample=1, Rounds Run=1181 | CV RMSE =0.0431 (Be
## Model 23 of 27, Settings: eta=0.1, max_depth=12, subsample=1, Rounds Run=145 | CV RMSE =0.0442 (Best
## Model 24 of 27, Settings: eta=0.3, max_depth=12, subsample=1, Rounds Run=64 | CV RMSE =0.0466 (Best
## Model 25 of 27, Settings: eta=0.01, max_depth=18, subsample=1, Rounds Run=1178 | CV RMSE =0.0431 (Be
```

```
## Model 26 of 27, Settings: eta=0.1, max_depth=18, subsample=1, Rounds Run=145 | CV RMSE =0.0442 (Best
## Model 27 of 27, Settings: eta=0.3, max_depth=18, subsample=1, Rounds Run=54 | CV RMSE =0.0468 (Best
```

```
# Summarize results
results_df2 <- do.call(rbind, lapply(results_list2, as.data.frame))
best_model_settings2 <- results_df2[which.min(results_df2$rmse),]

cat("\n--- Best Model Settings (Based on CV Logloss) ---\n")
```

```
##
## --- Best Model Settings (Based on CV Logloss) ---
```

```
print(best_model_settings2)
```

```
##    eta max_depth roundsrun subsample       rmse best_iteration
## 1 0.01         6       650       0.6 0.04205997            640
```

```
cat("---------------------------\n")
```

```
## ---------------------------
```

```
set.seed(123)
# Final Model with best settings
params_final2 <- list(
  booster = "gbtree",
  objective = "reg:squarederror",
  eta = best_model_settings2$eta,
  max_depth = best_model_settings2$max_depth,
  subsample = best_model_settings2$subsample,
  colsample_bytree = 0.8,
  nthread = 1,
  eval_metric = "rmse"
)

final_xgb_model2 <- xgboost(
  params = params_final2,
  data = train_matrix2,
  label = train_target2,
  nrounds = best_model_settings2$best_iteration,
  verbose = 1
)
```

```
## [1]  train-rmse:0.253737
## [2]  train-rmse:0.251273
## [3]  train-rmse:0.248844
## [4]  train-rmse:0.246427
## [5]  train-rmse:0.244038
## [6]  train-rmse:0.241667
## [7]  train-rmse:0.239329
## [8]  train-rmse:0.237009
## [9]  train-rmse:0.234715
## [10] train-rmse:0.232441
## [11] train-rmse:0.230220
## [12] train-rmse:0.228022
## [13] train-rmse:0.225821
## [14] train-rmse:0.223640
## [15] train-rmse:0.221473
```

```
## [16] train-rmse:0.219341
## [17] train-rmse:0.217211
## [18] train-rmse:0.215120
## [19] train-rmse:0.213032
## [20] train-rmse:0.210980
## [21] train-rmse:0.208949
## [22] train-rmse:0.206933
## [23] train-rmse:0.204939
## [24] train-rmse:0.202962
## [25] train-rmse:0.201021
## [26] train-rmse:0.199086
## [27] train-rmse:0.197172
## [28] train-rmse:0.195294
## [29] train-rmse:0.193438
## [30] train-rmse:0.191580
## [31] train-rmse:0.189732
## [32] train-rmse:0.187921
## [33] train-rmse:0.186114
## [34] train-rmse:0.184320
## [35] train-rmse:0.182557
## [36] train-rmse:0.180805
## [37] train-rmse:0.179071
## [38] train-rmse:0.177359
## [39] train-rmse:0.175641
## [40] train-rmse:0.173960
## [41] train-rmse:0.172285
## [42] train-rmse:0.170646
## [43] train-rmse:0.169005
## [44] train-rmse:0.167389
## [45] train-rmse:0.165794
## [46] train-rmse:0.164197
## [47] train-rmse:0.162619
## [48] train-rmse:0.161067
## [49] train-rmse:0.159526
## [50] train-rmse:0.158007
## [51] train-rmse:0.156496
## [52] train-rmse:0.155003
## [53] train-rmse:0.153527
## [54] train-rmse:0.152062
## [55] train-rmse:0.150614
## [56] train-rmse:0.149181
## [57] train-rmse:0.147761
## [58] train-rmse:0.146344
## [59] train-rmse:0.144948
## [60] train-rmse:0.143571
## [61] train-rmse:0.142227
## [62] train-rmse:0.140881
## [63] train-rmse:0.139545
## [64] train-rmse:0.138221
## [65] train-rmse:0.136913
## [66] train-rmse:0.135602
## [67] train-rmse:0.134319
## [68] train-rmse:0.133061
## [69] train-rmse:0.131801
```

```
## [70] train-rmse:0.130549
## [71] train-rmse:0.129314
## [72] train-rmse:0.128092
## [73] train-rmse:0.126898
## [74] train-rmse:0.125704
## [75] train-rmse:0.124514
## [76] train-rmse:0.123338
## [77] train-rmse:0.122177
## [78] train-rmse:0.121020
## [79] train-rmse:0.119888
## [80] train-rmse:0.118749
## [81] train-rmse:0.117635
## [82] train-rmse:0.116519
## [83] train-rmse:0.115418
## [84] train-rmse:0.114335
## [85] train-rmse:0.113258
## [86] train-rmse:0.112190
## [87] train-rmse:0.111141
## [88] train-rmse:0.110091
## [89] train-rmse:0.109057
## [90] train-rmse:0.108040
## [91] train-rmse:0.107024
## [92] train-rmse:0.106021
## [93] train-rmse:0.105014
## [94] train-rmse:0.104025
## [95] train-rmse:0.103054
## [96] train-rmse:0.102088
## [97] train-rmse:0.101129
## [98] train-rmse:0.100178
## [99] train-rmse:0.099240
## [100]    train-rmse:0.098317
## [101]    train-rmse:0.097399
## [102]    train-rmse:0.096492
## [103]    train-rmse:0.095589
## [104]    train-rmse:0.094699
## [105]    train-rmse:0.093812
## [106]    train-rmse:0.092940
## [107]    train-rmse:0.092070
## [108]    train-rmse:0.091220
## [109]    train-rmse:0.090373
## [110]    train-rmse:0.089549
## [111]    train-rmse:0.088719
## [112]    train-rmse:0.087901
## [113]    train-rmse:0.087085
## [114]    train-rmse:0.086276
## [115]    train-rmse:0.085477
## [116]    train-rmse:0.084690
## [117]    train-rmse:0.083906
## [118]    train-rmse:0.083140
## [119]    train-rmse:0.082371
## [120]    train-rmse:0.081607
## [121]    train-rmse:0.080864
## [122]    train-rmse:0.080118
## [123]    train-rmse:0.079373
```

```
## [124]    train-rmse:0.078643
## [125]    train-rmse:0.077925
## [126]    train-rmse:0.077207
## [127]    train-rmse:0.076494
## [128]    train-rmse:0.075789
## [129]    train-rmse:0.075095
## [130]    train-rmse:0.074405
## [131]    train-rmse:0.073729
## [132]    train-rmse:0.073057
## [133]    train-rmse:0.072395
## [134]    train-rmse:0.071734
## [135]    train-rmse:0.071077
## [136]    train-rmse:0.070437
## [137]    train-rmse:0.069795
## [138]    train-rmse:0.069154
## [139]    train-rmse:0.068524
## [140]    train-rmse:0.067903
## [141]    train-rmse:0.067283
## [142]    train-rmse:0.066674
## [143]    train-rmse:0.066076
## [144]    train-rmse:0.065479
## [145]    train-rmse:0.064890
## [146]    train-rmse:0.064306
## [147]    train-rmse:0.063724
## [148]    train-rmse:0.063160
## [149]    train-rmse:0.062600
## [150]    train-rmse:0.062034
## [151]    train-rmse:0.061470
## [152]    train-rmse:0.060920
## [153]    train-rmse:0.060374
## [154]    train-rmse:0.059829
## [155]    train-rmse:0.059287
## [156]    train-rmse:0.058757
## [157]    train-rmse:0.058230
## [158]    train-rmse:0.057715
## [159]    train-rmse:0.057193
## [160]    train-rmse:0.056678
## [161]    train-rmse:0.056174
## [162]    train-rmse:0.055674
## [163]    train-rmse:0.055186
## [164]    train-rmse:0.054695
## [165]    train-rmse:0.054218
## [166]    train-rmse:0.053736
## [167]    train-rmse:0.053260
## [168]    train-rmse:0.052790
## [169]    train-rmse:0.052325
## [170]    train-rmse:0.051864
## [171]    train-rmse:0.051411
## [172]    train-rmse:0.050968
## [173]    train-rmse:0.050517
## [174]    train-rmse:0.050080
## [175]    train-rmse:0.049644
## [176]    train-rmse:0.049215
## [177]    train-rmse:0.048777
```

```
## [178]    train-rmse:0.048359
## [179]    train-rmse:0.047939
## [180]    train-rmse:0.047529
## [181]    train-rmse:0.047123
## [182]    train-rmse:0.046716
## [183]    train-rmse:0.046313
## [184]    train-rmse:0.045916
## [185]    train-rmse:0.045514
## [186]    train-rmse:0.045127
## [187]    train-rmse:0.044748
## [188]    train-rmse:0.044372
## [189]    train-rmse:0.043990
## [190]    train-rmse:0.043606
## [191]    train-rmse:0.043239
## [192]    train-rmse:0.042866
## [193]    train-rmse:0.042505
## [194]    train-rmse:0.042139
## [195]    train-rmse:0.041784
## [196]    train-rmse:0.041429
## [197]    train-rmse:0.041082
## [198]    train-rmse:0.040731
## [199]    train-rmse:0.040391
## [200]    train-rmse:0.040046
## [201]    train-rmse:0.039709
## [202]    train-rmse:0.039379
## [203]    train-rmse:0.039044
## [204]    train-rmse:0.038715
## [205]    train-rmse:0.038392
## [206]    train-rmse:0.038066
## [207]    train-rmse:0.037749
## [208]    train-rmse:0.037433
## [209]    train-rmse:0.037121
## [210]    train-rmse:0.036818
## [211]    train-rmse:0.036514
## [212]    train-rmse:0.036214
## [213]    train-rmse:0.035918
## [214]    train-rmse:0.035626
## [215]    train-rmse:0.035328
## [216]    train-rmse:0.035028
## [217]    train-rmse:0.034742
## [218]    train-rmse:0.034465
## [219]    train-rmse:0.034180
## [220]    train-rmse:0.033906
## [221]    train-rmse:0.033636
## [222]    train-rmse:0.033361
## [223]    train-rmse:0.033095
## [224]    train-rmse:0.032833
## [225]    train-rmse:0.032568
## [226]    train-rmse:0.032307
## [227]    train-rmse:0.032041
## [228]    train-rmse:0.031790
## [229]    train-rmse:0.031540
## [230]    train-rmse:0.031300
## [231]    train-rmse:0.031049
```

```
## [232]    train-rmse:0.030798
## [233]    train-rmse:0.030560
## [234]    train-rmse:0.030320
## [235]    train-rmse:0.030078
## [236]    train-rmse:0.029840
## [237]    train-rmse:0.029603
## [238]    train-rmse:0.029379
## [239]    train-rmse:0.029152
## [240]    train-rmse:0.028925
## [241]    train-rmse:0.028701
## [242]    train-rmse:0.028476
## [243]    train-rmse:0.028257
## [244]    train-rmse:0.028032
## [245]    train-rmse:0.027819
## [246]    train-rmse:0.027605
## [247]    train-rmse:0.027397
## [248]    train-rmse:0.027189
## [249]    train-rmse:0.026981
## [250]    train-rmse:0.026778
## [251]    train-rmse:0.026577
## [252]    train-rmse:0.026379
## [253]    train-rmse:0.026181
## [254]    train-rmse:0.025988
## [255]    train-rmse:0.025800
## [256]    train-rmse:0.025612
## [257]    train-rmse:0.025418
## [258]    train-rmse:0.025227
## [259]    train-rmse:0.025055
## [260]    train-rmse:0.024877
## [261]    train-rmse:0.024693
## [262]    train-rmse:0.024515
## [263]    train-rmse:0.024338
## [264]    train-rmse:0.024159
## [265]    train-rmse:0.023988
## [266]    train-rmse:0.023829
## [267]    train-rmse:0.023657
## [268]    train-rmse:0.023487
## [269]    train-rmse:0.023320
## [270]    train-rmse:0.023150
## [271]    train-rmse:0.022987
## [272]    train-rmse:0.022831
## [273]    train-rmse:0.022671
## [274]    train-rmse:0.022514
## [275]    train-rmse:0.022353
## [276]    train-rmse:0.022203
## [277]    train-rmse:0.022049
## [278]    train-rmse:0.021905
## [279]    train-rmse:0.021748
## [280]    train-rmse:0.021602
## [281]    train-rmse:0.021447
## [282]    train-rmse:0.021299
## [283]    train-rmse:0.021157
## [284]    train-rmse:0.021024
## [285]    train-rmse:0.020885
```

```
## [286]    train-rmse:0.020743
## [287]    train-rmse:0.020606
## [288]    train-rmse:0.020471
## [289]    train-rmse:0.020338
## [290]    train-rmse:0.020203
## [291]    train-rmse:0.020070
## [292]    train-rmse:0.019940
## [293]    train-rmse:0.019809
## [294]    train-rmse:0.019680
## [295]    train-rmse:0.019551
## [296]    train-rmse:0.019421
## [297]    train-rmse:0.019304
## [298]    train-rmse:0.019191
## [299]    train-rmse:0.019070
## [300]    train-rmse:0.018943
## [301]    train-rmse:0.018823
## [302]    train-rmse:0.018708
## [303]    train-rmse:0.018593
## [304]    train-rmse:0.018479
## [305]    train-rmse:0.018361
## [306]    train-rmse:0.018252
## [307]    train-rmse:0.018139
## [308]    train-rmse:0.018027
## [309]    train-rmse:0.017924
## [310]    train-rmse:0.017819
## [311]    train-rmse:0.017710
## [312]    train-rmse:0.017607
## [313]    train-rmse:0.017503
## [314]    train-rmse:0.017406
## [315]    train-rmse:0.017308
## [316]    train-rmse:0.017208
## [317]    train-rmse:0.017107
## [318]    train-rmse:0.017016
## [319]    train-rmse:0.016918
## [320]    train-rmse:0.016818
## [321]    train-rmse:0.016729
## [322]    train-rmse:0.016637
## [323]    train-rmse:0.016550
## [324]    train-rmse:0.016461
## [325]    train-rmse:0.016374
## [326]    train-rmse:0.016286
## [327]    train-rmse:0.016195
## [328]    train-rmse:0.016107
## [329]    train-rmse:0.016021
## [330]    train-rmse:0.015944
## [331]    train-rmse:0.015862
## [332]    train-rmse:0.015781
## [333]    train-rmse:0.015693
## [334]    train-rmse:0.015613
## [335]    train-rmse:0.015527
## [336]    train-rmse:0.015448
## [337]    train-rmse:0.015371
## [338]    train-rmse:0.015289
## [339]    train-rmse:0.015211
```

```
## [340]    train-rmse:0.015138
## [341]    train-rmse:0.015066
## [342]    train-rmse:0.014990
## [343]    train-rmse:0.014916
## [344]    train-rmse:0.014839
## [345]    train-rmse:0.014760
## [346]    train-rmse:0.014686
## [347]    train-rmse:0.014612
## [348]    train-rmse:0.014547
## [349]    train-rmse:0.014471
## [350]    train-rmse:0.014397
## [351]    train-rmse:0.014329
## [352]    train-rmse:0.014265
## [353]    train-rmse:0.014203
## [354]    train-rmse:0.014134
## [355]    train-rmse:0.014067
## [356]    train-rmse:0.014011
## [357]    train-rmse:0.013943
## [358]    train-rmse:0.013879
## [359]    train-rmse:0.013824
## [360]    train-rmse:0.013767
## [361]    train-rmse:0.013711
## [362]    train-rmse:0.013647
## [363]    train-rmse:0.013582
## [364]    train-rmse:0.013520
## [365]    train-rmse:0.013458
## [366]    train-rmse:0.013401
## [367]    train-rmse:0.013347
## [368]    train-rmse:0.013294
## [369]    train-rmse:0.013236
## [370]    train-rmse:0.013179
## [371]    train-rmse:0.013123
## [372]    train-rmse:0.013069
## [373]    train-rmse:0.013013
## [374]    train-rmse:0.012956
## [375]    train-rmse:0.012907
## [376]    train-rmse:0.012867
## [377]    train-rmse:0.012817
## [378]    train-rmse:0.012764
## [379]    train-rmse:0.012709
## [380]    train-rmse:0.012662
## [381]    train-rmse:0.012610
## [382]    train-rmse:0.012557
## [383]    train-rmse:0.012504
## [384]    train-rmse:0.012461
## [385]    train-rmse:0.012417
## [386]    train-rmse:0.012371
## [387]    train-rmse:0.012320
## [388]    train-rmse:0.012270
## [389]    train-rmse:0.012232
## [390]    train-rmse:0.012182
## [391]    train-rmse:0.012142
## [392]    train-rmse:0.012097
## [393]    train-rmse:0.012055
```

```
## [394]    train-rmse:0.012022
## [395]    train-rmse:0.011977
## [396]    train-rmse:0.011935
## [397]    train-rmse:0.011892
## [398]    train-rmse:0.011850
## [399]    train-rmse:0.011810
## [400]    train-rmse:0.011771
## [401]    train-rmse:0.011730
## [402]    train-rmse:0.011695
## [403]    train-rmse:0.011660
## [404]    train-rmse:0.011625
## [405]    train-rmse:0.011589
## [406]    train-rmse:0.011554
## [407]    train-rmse:0.011518
## [408]    train-rmse:0.011482
## [409]    train-rmse:0.011441
## [410]    train-rmse:0.011411
## [411]    train-rmse:0.011378
## [412]    train-rmse:0.011338
## [413]    train-rmse:0.011309
## [414]    train-rmse:0.011271
## [415]    train-rmse:0.011240
## [416]    train-rmse:0.011206
## [417]    train-rmse:0.011172
## [418]    train-rmse:0.011144
## [419]    train-rmse:0.011107
## [420]    train-rmse:0.011082
## [421]    train-rmse:0.011051
## [422]    train-rmse:0.011015
## [423]    train-rmse:0.010986
## [424]    train-rmse:0.010957
## [425]    train-rmse:0.010924
## [426]    train-rmse:0.010889
## [427]    train-rmse:0.010855
## [428]    train-rmse:0.010823
## [429]    train-rmse:0.010798
## [430]    train-rmse:0.010771
## [431]    train-rmse:0.010737
## [432]    train-rmse:0.010707
## [433]    train-rmse:0.010684
## [434]    train-rmse:0.010649
## [435]    train-rmse:0.010614
## [436]    train-rmse:0.010582
## [437]    train-rmse:0.010555
## [438]    train-rmse:0.010535
## [439]    train-rmse:0.010502
## [440]    train-rmse:0.010473
## [441]    train-rmse:0.010442
## [442]    train-rmse:0.010414
## [443]    train-rmse:0.010378
## [444]    train-rmse:0.010357
## [445]    train-rmse:0.010326
## [446]    train-rmse:0.010293
## [447]    train-rmse:0.010271
```

```
## [448]    train-rmse:0.010246
## [449]    train-rmse:0.010220
## [450]    train-rmse:0.010189
## [451]    train-rmse:0.010171
## [452]    train-rmse:0.010145
## [453]    train-rmse:0.010120
## [454]    train-rmse:0.010098
## [455]    train-rmse:0.010072
## [456]    train-rmse:0.010053
## [457]    train-rmse:0.010030
## [458]    train-rmse:0.010007
## [459]    train-rmse:0.009985
## [460]    train-rmse:0.009959
## [461]    train-rmse:0.009931
## [462]    train-rmse:0.009908
## [463]    train-rmse:0.009883
## [464]    train-rmse:0.009853
## [465]    train-rmse:0.009834
## [466]    train-rmse:0.009816
## [467]    train-rmse:0.009798
## [468]    train-rmse:0.009772
## [469]    train-rmse:0.009752
## [470]    train-rmse:0.009727
## [471]    train-rmse:0.009707
## [472]    train-rmse:0.009693
## [473]    train-rmse:0.009667
## [474]    train-rmse:0.009640
## [475]    train-rmse:0.009617
## [476]    train-rmse:0.009593
## [477]    train-rmse:0.009573
## [478]    train-rmse:0.009551
## [479]    train-rmse:0.009533
## [480]    train-rmse:0.009509
## [481]    train-rmse:0.009486
## [482]    train-rmse:0.009465
## [483]    train-rmse:0.009451
## [484]    train-rmse:0.009432
## [485]    train-rmse:0.009413
## [486]    train-rmse:0.009394
## [487]    train-rmse:0.009373
## [488]    train-rmse:0.009351
## [489]    train-rmse:0.009335
## [490]    train-rmse:0.009317
## [491]    train-rmse:0.009301
## [492]    train-rmse:0.009282
## [493]    train-rmse:0.009266
## [494]    train-rmse:0.009246
## [495]    train-rmse:0.009233
## [496]    train-rmse:0.009210
## [497]    train-rmse:0.009190
## [498]    train-rmse:0.009175
## [499]    train-rmse:0.009158
## [500]    train-rmse:0.009138
## [501]    train-rmse:0.009122
```

```
## [502]    train-rmse:0.009104
## [503]    train-rmse:0.009087
## [504]    train-rmse:0.009070
## [505]    train-rmse:0.009052
## [506]    train-rmse:0.009038
## [507]    train-rmse:0.009028
## [508]    train-rmse:0.009013
## [509]    train-rmse:0.008998
## [510]    train-rmse:0.008980
## [511]    train-rmse:0.008965
## [512]    train-rmse:0.008948
## [513]    train-rmse:0.008932
## [514]    train-rmse:0.008917
## [515]    train-rmse:0.008901
## [516]    train-rmse:0.008888
## [517]    train-rmse:0.008869
## [518]    train-rmse:0.008857
## [519]    train-rmse:0.008841
## [520]    train-rmse:0.008827
## [521]    train-rmse:0.008813
## [522]    train-rmse:0.008798
## [523]    train-rmse:0.008776
## [524]    train-rmse:0.008762
## [525]    train-rmse:0.008743
## [526]    train-rmse:0.008728
## [527]    train-rmse:0.008716
## [528]    train-rmse:0.008701
## [529]    train-rmse:0.008684
## [530]    train-rmse:0.008669
## [531]    train-rmse:0.008655
## [532]    train-rmse:0.008641
## [533]    train-rmse:0.008624
## [534]    train-rmse:0.008616
## [535]    train-rmse:0.008603
## [536]    train-rmse:0.008593
## [537]    train-rmse:0.008580
## [538]    train-rmse:0.008563
## [539]    train-rmse:0.008550
## [540]    train-rmse:0.008535
## [541]    train-rmse:0.008526
## [542]    train-rmse:0.008505
## [543]    train-rmse:0.008490
## [544]    train-rmse:0.008472
## [545]    train-rmse:0.008458
## [546]    train-rmse:0.008446
## [547]    train-rmse:0.008439
## [548]    train-rmse:0.008426
## [549]    train-rmse:0.008413
## [550]    train-rmse:0.008401
## [551]    train-rmse:0.008390
## [552]    train-rmse:0.008381
## [553]    train-rmse:0.008371
## [554]    train-rmse:0.008364
## [555]    train-rmse:0.008356
```

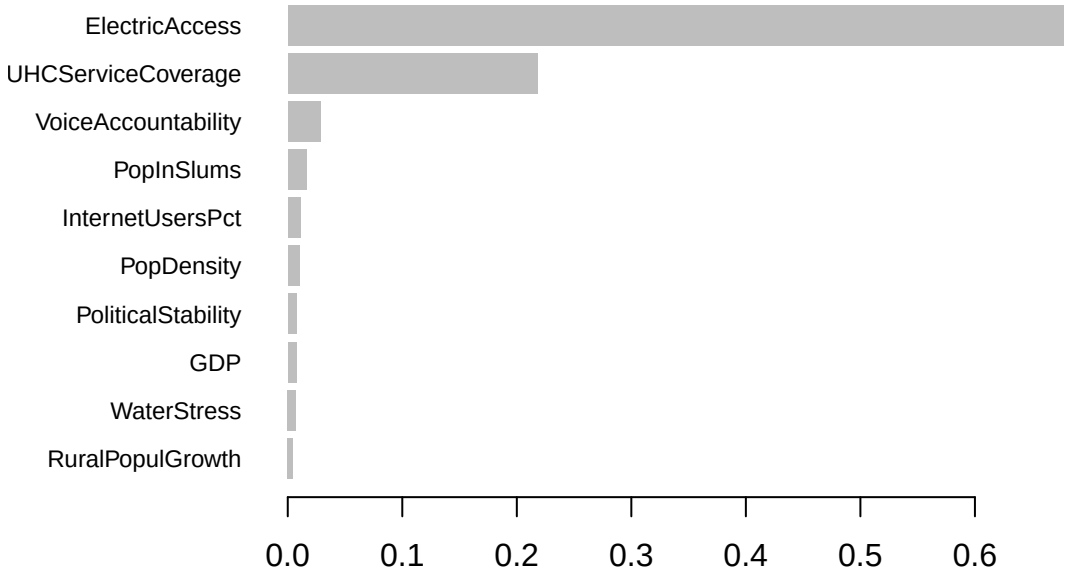
```
## [556]    train-rmse:0.008343
## [557]    train-rmse:0.008330
## [558]    train-rmse:0.008316
## [559]    train-rmse:0.008304
## [560]    train-rmse:0.008290
## [561]    train-rmse:0.008276
## [562]    train-rmse:0.008268
## [563]    train-rmse:0.008255
## [564]    train-rmse:0.008245
## [565]    train-rmse:0.008235
## [566]    train-rmse:0.008226
## [567]    train-rmse:0.008214
## [568]    train-rmse:0.008200
## [569]    train-rmse:0.008189
## [570]    train-rmse:0.008177
## [571]    train-rmse:0.008163
## [572]    train-rmse:0.008154
## [573]    train-rmse:0.008142
## [574]    train-rmse:0.008127
## [575]    train-rmse:0.008120
## [576]    train-rmse:0.008109
## [577]    train-rmse:0.008099
## [578]    train-rmse:0.008095
## [579]    train-rmse:0.008090
## [580]    train-rmse:0.008083
## [581]    train-rmse:0.008075
## [582]    train-rmse:0.008063
## [583]    train-rmse:0.008057
## [584]    train-rmse:0.008046
## [585]    train-rmse:0.008038
## [586]    train-rmse:0.008028
## [587]    train-rmse:0.008020
## [588]    train-rmse:0.008011
## [589]    train-rmse:0.008005
## [590]    train-rmse:0.007995
## [591]    train-rmse:0.007984
## [592]    train-rmse:0.007972
## [593]    train-rmse:0.007969
## [594]    train-rmse:0.007964
## [595]    train-rmse:0.007950
## [596]    train-rmse:0.007936
## [597]    train-rmse:0.007922
## [598]    train-rmse:0.007916
## [599]    train-rmse:0.007903
## [600]    train-rmse:0.007888
## [601]    train-rmse:0.007878
## [602]    train-rmse:0.007870
## [603]    train-rmse:0.007853
## [604]    train-rmse:0.007843
## [605]    train-rmse:0.007839
## [606]    train-rmse:0.007825
## [607]    train-rmse:0.007818
## [608]    train-rmse:0.007807
## [609]    train-rmse:0.007804
```

```
## [610]    train-rmse:0.007793
## [611]    train-rmse:0.007783
## [612]    train-rmse:0.007769
## [613]    train-rmse:0.007755
## [614]    train-rmse:0.007748
## [615]    train-rmse:0.007740
## [616]    train-rmse:0.007732
## [617]    train-rmse:0.007722
## [618]    train-rmse:0.007711
## [619]    train-rmse:0.007700
## [620]    train-rmse:0.007686
## [621]    train-rmse:0.007679
## [622]    train-rmse:0.007675
## [623]    train-rmse:0.007662
## [624]    train-rmse:0.007651
## [625]    train-rmse:0.007641
## [626]    train-rmse:0.007630
## [627]    train-rmse:0.007622
## [628]    train-rmse:0.007613
## [629]    train-rmse:0.007610
## [630]    train-rmse:0.007605
## [631]    train-rmse:0.007594
## [632]    train-rmse:0.007591
## [633]    train-rmse:0.007584
## [634]    train-rmse:0.007574
## [635]    train-rmse:0.007564
## [636]    train-rmse:0.007555
## [637]    train-rmse:0.007551
## [638]    train-rmse:0.007546
## [639]    train-rmse:0.007536
## [640]    train-rmse:0.007527
```

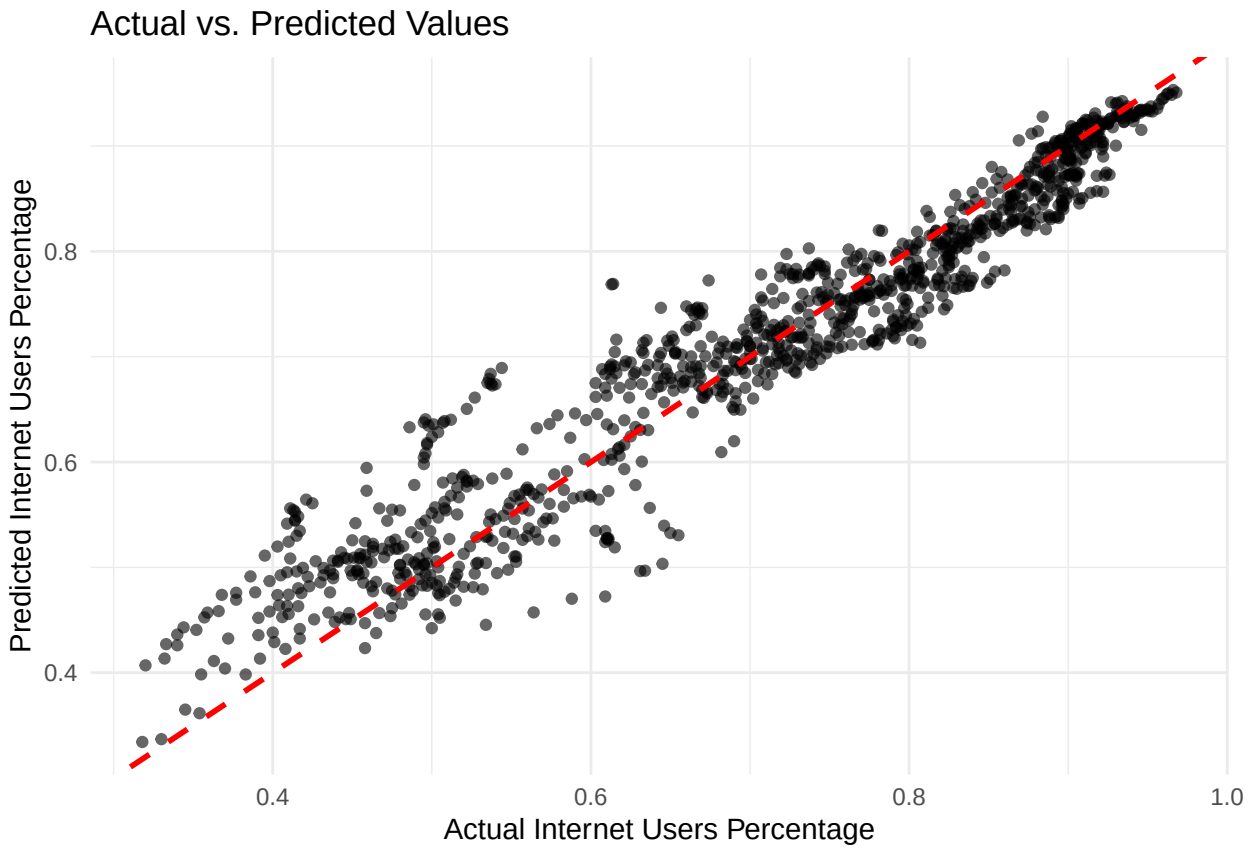
```r
# Make predictions on the test data
predictions2 <- predict(final_xgb_model2, newdata = dtest2)

# Calculate the Mean Absolute Error to evaluate the model's performance
mean_absolute_error2 <- mean(abs(test_target2 - predictions2))
cat("Mean Absolute Error:", mean_absolute_error2, "\n")
```

```
## Mean Absolute Error: 0.03533873
```

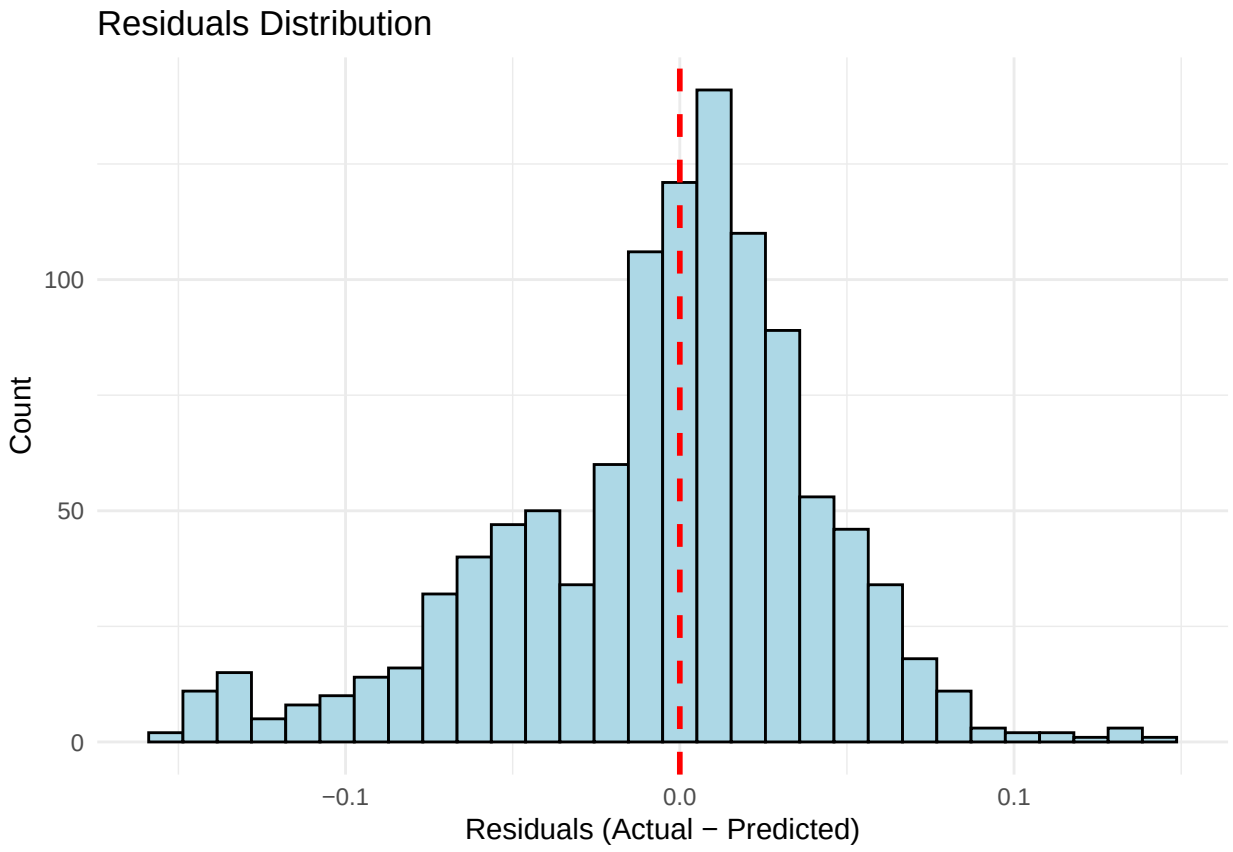
```r
# Visualize Feature Importance

importance_matrix2 <- xgb.importance(feature_names = colnames(dtrain2),
                                     model = final_xgb_model2)

# Plot the top 10 most important features
xgb.plot.importance(importance_matrix2, top_n = 10)
```




```r
# Get feature importance from the model
importance_matrix2 <- xgb.importance(feature_names = colnames(dtrain2),
                                     model = final_xgb_model2)


# Create a data frame for plotting predictions vs. actuals
results2 <- data.frame(Actual = test_target2, Predicted = predictions2)

# Create a scatter plot of Actual vs. Predicted values
xg_scatter2 <- ggplot(results2, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.6) +
  geom_abline(intercept = 0, slope = 1, color = "red",
              linetype = "dashed", linewidth = 1) +
  labs(
    title = "Actual vs. Predicted Values",
    x = "Actual Internet Users Percentage",
    y = "Predicted Internet Users Percentage"
  ) +
  theme_minimal()

print(xg_scatter2)
```




```r
# Create a residual plot
results2$Residuals <- results2$Actual - results2$Predicted
xg_resid2 <- ggplot(results2, aes(x = Residuals)) +
  geom_histogram(bins = 30, fill = "lightblue", color = "black") +
  geom_vline(xintercept = 0, color = "red", linetype = "dashed", linewidth = 1) +
  labs(
    title = "Residuals Distribution",
    x = "Residuals (Actual - Predicted)",
    y = "Count"
  ) +
  theme_minimal()

print(xg_resid2)
```


Residuals Distribution
Count
100
50
0
−0.1
0.0
0.1
Residuals (Actual − Predicted)